

Worksheet Tebimbing PBO (Pekan 14)

Studi Kasus: Sistem Manajemen Kantin Kampus

Deskripsi Kasus

Kamu adalah seorang programmer yang diminta untuk membangun program sederhana berbasis Java yang memodelkan sistem kantin kampus. Kantin ini menjual berbagai jenis menu makanan dan minuman. Setiap produk yang dijual memiliki nama, harga, dan stok. Pembeli bisa memesan beberapa produk sekaligus, dan kasir akan menghitung total belanja beserta diskon (jika ada).

Persyaratan Teknis

Struktur Class yang Harus Dibuat

Kantin

```
|  
|— interface Diskonable  
|— abstract class Produk  
|   |— class Makanan (extends Produk, implements Diskonable)  
|   |— class Minuman (extends Produk)  
|— class Transaksi
```

Detail Setiap Class

1. Interface **Diskonable**

Interface ini merepresentasikan kontrak bahwa suatu produk **bisa diberikan diskon**.

Method yang wajib diimplementasikan:

Method	Return Type	Keterangan
<code>hitungDiskon()</code>	<code>double</code>	Mengembalikan nilai diskon dalam rupiah
<code>getHargaSetelahDiskon()</code>	<code>double</code>	Mengembalikan harga setelah dikurangi diskon

2. Abstract Class `Produk`

Class abstrak yang menjadi cetak biru untuk semua produk di kantin.

Atribut (semua `protected`):

Atribut	Tipe	Keterangan
<code>nama</code>	<code>String</code>	Nama produk
<code>harga</code>	<code>double</code>	Harga satuan
<code>stok</code>	<code>int</code>	Jumlah stok tersedia

Method:

Method	Keterangan
Constructor <code>Produk(nama, harga, stok)</code>	Menginisialisasi atribut
<code>getNama()</code>	Getter untuk nama
<code>getHarga()</code>	Getter untuk harga
<code>getStok()</code>	Getter untuk stok
<code>kurangiStok(int jumlah)</code>	Mengurangi stok; tampilkan pesan error jika stok tidak cukup

Method	Keterangan
<code>abstract String getKategori()</code>	Mengembalikan kategori produk (wajib diisi subclass)
<code>tampilkanInfo()</code>	Mencetak informasi produk ke konsol

Catatan: `getKategori()` adalah method abstrak — setiap subclass **wajib** mengimplementasikannya.

3. Class **Makanan** (extends **Produk**, implements **Diskonable**)

Merepresentasikan produk makanan. Makanan mendapat **diskon 10%** jika harganya di atas Rp 15.000.

Atribut tambahan:

Atribut	Tipe	Keterangan
<code>jenisMenu</code>	<code>String</code>	Contoh: "Nasi", "Mie", "Snack"

Method yang wajib diimplementasikan:

- `getKategori()` → mengembalikan "Makanan"
- `hitungDiskon()` → 10% dari harga jika harga > 15000, selain itu 0
- `getHargaSetelahDiskon()` → harga dikurangi hasil `hitungDiskon()`
- Override `tampilkanInfo()` → tampilkan info produk + info diskon

4. Class **Minuman** (extends **Produk**)

Merepresentasikan produk minuman. Minuman **tidak mendapat diskon**.

Atribut tambahan:

Atribut	Tipe	Keterangan
<code>ukuranMl</code>	<code>int</code>	Volume minuman dalam mililiter

Method yang wajib diimplementasikan:

- `getKategori()` → mengembalikan "Minuman"
- Override `tampilkanInfo()` → tampilkan info produk + ukuran dalam mL

5. Class Transaksi

Merepresentasikan satu transaksi pembelian seorang pelanggan.

Atribut:

Atribut	Tipe	Keterangan
<code>namaPelanggan</code>	<code>String</code>	Nama pembeli
<code>daftarProduk</code>	<code>ArrayList<Produk></code>	Daftar produk yang dibeli
<code>jumlahPesanan</code>	<code>ArrayList<Integer></code>	Jumlah masing-masing produk

Method:

Method	Keterangan
Constructor <code>Transaksi(namaPelanggan)</code>	Inisialisasi atribut
<code>tambahProduk(Produk p, int jumlah)</code>	Menambahkan produk ke transaksi; panggil <code>kurangiStok()</code>
<code>hitungTotal()</code>	Menghitung total harga: gunakan <code>getHargaSetelahDiskon()</code> untuk Makanan, dan <code>getHarga()</code> untuk Minuman
<code>cetakStruk()</code>	Mencetak struk pembelian ke konsol (lihat format di bawah)

Format Struk yang Diharapkan

=====

KANTIN KAMPUS SEJAHTERA

=====

Pelanggan : Budi Santoso

No	Produk	Qty	Harga
1.	Nasi Goreng Spesial	2	Rp 24.000 *
2.	Es Teh Manis	1	Rp 5.000

* harga setelah diskon

TOTAL : Rp 53.000

=====

Terima kasih! 😊

=====

Tanda * muncul pada produk yang mendapat diskon.

Instruksi Pengerjaan (dipandu oleh asisten)

Langkah 1 – Buat Interface

Buat file `Diskonable.java` dan definisikan dua method-nya.

Langkah 2 – Buat Abstract Class

Buat file `Produk.java`. Pastikan `getKategori()` dideklarasikan sebagai `abstract`.

Langkah 3 – Buat Subclass

Buat `Makanan.java` dan `Minuman.java`. Perhatikan:

- `Makanan` harus `extends Produk` **dan** `implements Diskonable`
- `Minuman` hanya `extends Produk`

Langkah 4 – Buat Class Transaksi

Buat `Transaksi.java`. Gunakan `instanceof` untuk mengecek apakah produk adalah `Makanan` saat menghitung total.

Langkah 5 – Buat Program Utama

Buat `Main.java` dan jalankan skenario berikut:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        // 1. Buat produk  
  
        Makanan nasiGoreng = new Makanan("Nasi Goreng Spesial", 12000, 10, "Nasi");  
  
        Makanan ayamGoreng = new Makanan("Ayam Goreng Crispy", 18000, 5, "Lauk");  
  
        Minuman esTeh = new Minuman("Es Teh Manis", 5000, 20, 350);  
  
        Minuman jusAlpukat = new Minuman("Jus Alpukat", 12000, 8, 250);  
  
        // 2. Tampilkan semua produk  
  
        System.out.println("=== MENU KANTIN ===");  
  
        nasiGoreng.tampilkanInfo();  
  
        ayamGoreng.tampilkanInfo();  
  
        esTeh.tampilkanInfo();  
  
        jusAlpukat.tampilkanInfo();  
    }  
}
```

```
// 3. Buat transaksi

Transaksi t1 = new Transaksi("Budi Santoso");

t1.tambahProduk(ayamGoreng, 2);

t1.tambahProduk(esTeh, 1);

t1.cetakStruk();

Transaksi t2 = new Transaksi("Sari Dewi");

t2.tambahProduk(nasiGoreng, 1);

t2.tambahProduk(jusAlpukat, 2);

t2.cetakStruk();

}

}
```

Pertanyaan Refleksi

Jawab pertanyaan berikut dalam komentar di bagian atas file `Main.java`:

1. Mengapa `Produk` dibuat sebagai **abstract class** dan bukan class biasa?
2. Apa perbedaan menggunakan **interface** `Diskonable` dibandingkan langsung menulis method diskon di class `Makanan`?
3. Sebutkan **di mana** konsep **polymorphism** terjadi dalam program kamu!